

Memory Model in Algorithms

Stack Frames, Heap Memory, Pointers & References

1. Time & Space Complexity: The Foundation

In algorithmic analysis, **time complexity** measures how execution time grows with input size, and **space complexity** measures how memory usage grows with input size. These two are often in tension — making an algorithm faster frequently requires storing more data, and reducing memory usage often means recomputing values instead of caching them. This is the **space-time tradeoff**.

To understand where an algorithm's memory actually goes during runtime, we need to understand two distinct regions of memory: the **stack** and the **heap**.

2. Stack vs Heap: The Two Memory Regions

The Stack

The stack is a region of memory that grows and shrinks in a strict Last-In-First-Out order. Every time a function is called, a new **stack frame** is pushed onto the stack. This frame holds:

- Function parameters
- Local variables
- The return address (where to jump back after the function finishes)
- Bookkeeping data the CPU needs

When the function returns, its stack frame is destroyed and all its local data disappears.

The Heap

The heap is a large, flexible pool of memory used for data that is too big for the stack, has an unknown size at compile time, or needs to outlive the function that created it. Memory on the heap is allocated explicitly (via `new`, `malloc`, or automatically when creating objects in languages like Python/Java) and lives until it is either freed manually or garbage collected.

Visual Comparison

STACK	HEAP
(fast, auto-managed)	(flexible, dynamic)
- Function frames	- Arrays, objects
- Local variables	- Data structures
- Return addresses	- Anything from new/ malloc
- LIFO order	- Freed manually or by GC
- Freed automatically	

3. How an Algorithm Consumes Space at Runtime

Let us take **Merge Sort** as an example and trace exactly what physically occupies memory while it runs on the input [8, 3, 5, 1].

The Algorithm

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

What Takes Up Space During Execution

(a) Stack Frames — Call Stack Memory

Every recursive call creates a new stack frame holding parameters (arr), local variables (mid, left, right), and the return address. Frames stack up until the innermost call returns.

(b) Sub-array Allocations — Heap Memory

arr[:mid] and arr[mid:] allocate **brand-new arrays** on the heap. At the deepest recursion level, many intermediate arrays coexist until they get merged and released.

(c) The result Buffer in merge()

Each merge() call allocates a fresh list on the heap that grows dynamically as elements are appended.

(d) Loop Counters and Temporary Variables

Variables like i, j, mid occupy small fixed slots in each stack frame. Small individually, but they accumulate across recursive calls.

Memory Snapshot at Deepest Recursion

CALL STACK (grows downward)	HEAP
+-----+	+-----+
merge_sort([8,3,5,1])	--ref--> [8, 3, 5, 1]
mid=2, left=?, right=?	+-----+
+-----+	[8, 3]
merge_sort([8,3])	--ref-->+-----+
mid=1, left=?, right=?	[5, 1]
+-----+	+-----+
merge_sort([8])	--ref--> [8]

| returns [8] | +-----+
+-----+

The stack holds **execution context**; the heap holds the **data being processed**. Both consume memory simultaneously during runtime. Total auxiliary space: **$O(n)$** .

4. What is a Reference?

That `--ref-->` arrow between stack and heap represents a **reference** (also called a pointer in lower-level languages). A reference is essentially a **memory address** — a number that tells the computer *where* to find the actual data.

Why We Need References

Small, fixed-size data (integers, booleans) fits directly on the stack. But large or variable-size data (arrays, objects, lists) is allocated on the heap, and the stack just holds a small **reference** (typically 8 bytes on a 64-bit machine) that points to the actual data.

Stack:		Heap:
+-----+		+-----+
arr --ref-->	----->	[8, 3, 5, 1]
+-----+		+-----+
(8 bytes)		(32+ bytes)

Analogy

Think of your desk (the stack) as small and tidy. You need to work with a giant document (the array). Instead of piling the whole document on your desk, you keep a sticky note with a filing-cabinet drawer number on your desk. The document sits in the filing cabinet (the heap), and your sticky note points to it.

5. What is a Pointer?

A **pointer** is a variable whose value is a memory address. It is the same underlying concept as a reference, but the term is used more explicitly in lower-level languages like C and C++, where addresses can be manipulated directly (arithmetic, reassignment, null checking).

Example in C

```
int x = 42;           // x holds the value 42
int *p = &x;          // p holds the ADDRESS of x
                      // *p means "the value at address p" = 42
```

Does a Pointer Bypass the Stack Frame?

No. The pointer itself is a variable and must live somewhere — usually on the stack (as a local variable), sometimes inside another heap object (like a linked-list node), or in a CPU register. What a pointer *does* allow is **direct access to heap memory** once you have the address: the CPU jumps straight to that address in a single instruction. The stack frame is not a checkpoint the data passes through — it just *stores the address label*.

Stack Frame (myFunction):		Heap:
+-----+		+-----+
ptr = 0x9A00	----->	0x9A00: [_, _, _, ...]
+-----+		100 ints
		+-----+

6. Pointer vs Normal Variable vs Reference Variable

Quick Definitions

Type	What It Holds	Analogy
Normal Variable	An actual value	A box containing an item
Pointer	A memory address (mutable, nullable)	A sticky note with an address you can erase and rewrite
Reference	An alias for another variable (fixed)	A permanent nickname tied to a person

Feature-by-Feature Comparison

Feature	Normal Variable	Pointer	Reference
Holds a value?	Yes, directly	No, holds address	No, aliases another
Must initialize?	No	No (but should)	Yes, mandatory
Can reassign?	Yes	Yes	No
Can be null?	N/A	Yes	No
Dereferencing?	Not needed	Yes (*p)	Not needed
Arithmetic?	Yes (if numeric)	Yes (p++)	No
Own memory slot?	Yes	Yes (holds address)	Usually optimized away

Combined Example

```
int x = 5;

int y = x;      // NORMAL VARIABLE: y is a copy
int *p = &x;    // POINTER: p holds x's address
int &r = x;     // REFERENCE: r is an alias for x

y = 100;        // x is unchanged (y is independent)
*p = 200;       // x becomes 200 (pointer modifies via address)
r = 300;        // x becomes 300 (reference IS x)
```

7. Stack & Heap View of All Three

Normal Variable — Lives Entirely in the Stack Frame

```
void myFunction() {  
    int x = 42;  
}
```

STACK FRAME:	HEAP:
+-----+	+-----+
x = 42	(nothing here)
+-----+	+-----+

The value **42** sits directly inside the stack frame. When the function returns, the frame is destroyed and x vanishes with it.

Pointer — Address on Stack, Data on Heap

```
void myFunction() {  
    int *p = new int(42);  
}
```

STACK FRAME:	HEAP:
+-----+	+-----+
p = 0x9A00 --ref--> 0x9A00: 42	
+-----+	+-----+

Two separate allocations exist. When the function returns, p is destroyed, but the heap value **persists** — and becomes a memory leak in C++ unless explicitly freed with delete.

Reference — Just an Alias in the Same Frame

```
void myFunction() {  
    int x = 42;  
    int &r = x;  
}
```

STACK FRAME:	HEAP:
+-----+	+-----+
x = 42 (also called r)	(nothing here)
+-----+	+-----+

There is just **one slot**. The name r is a second label pointing to the exact same memory location as x. Assigning to r writes to the same slot as x.

All Three Together

```
void myFunction() {  
    int x = 42;           // normal  
    int *p = new int(99); // pointer to heap  
    int &r = x;           // reference to x  
}
```

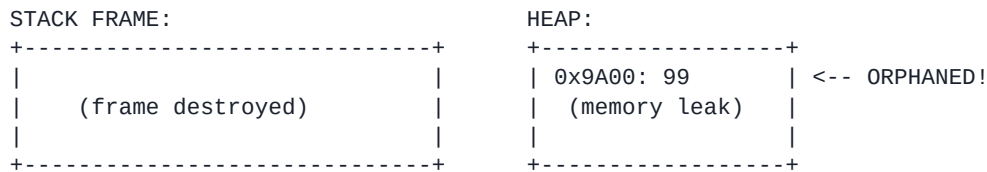
STACK FRAME:	HEAP:
+-----+	+-----+
x = 42 (also called r)	0x9A00: 99
p = 0x9A00	--ref>

+-----+

+-----+

8. What Happens When the Function Returns

The stack frame is torn down completely:



- **x** → gone (was in stack frame)
- **r** → gone (was just a label for x)
- **p** → gone (was in stack frame)
- **Heap allocation 99** → still there, but unreachable → memory leak (unless GC'd)

This is exactly why C++ programmers must call `delete p;` before the function ends — to free that heap memory manually. Languages like Java and Python handle this automatically through garbage collection.

9. Space Complexity Summary

Connecting everything back to algorithmic space complexity:

Variable Type	Stack Space	Heap Space
Normal variable (int x)	Size of the value (e.g., 4 bytes)	0
Pointer (int *p)	Size of an address (8 bytes on 64-bit)	Whatever was allocated via <code>new</code> / <code>malloc</code>
Reference (int &r)	0 (usually optimized)	0 (unless it aliases heap data)

When Analyzing an Algorithm's Space Usage, Look For:

- **Recursive calls** → each call pushes a stack frame; local variables and pointers accumulate on the stack
- **Dynamic allocations** (`new`/`malloc`, list/dict creation) → grow the heap; persist across recursive calls until freed or garbage collected
- **References/aliases** → essentially free; they do not allocate, they just relabel existing memory

10. The Mental Model

Picture the stack frame as a **whiteboard on your desk** and the heap as a **warehouse across town**:

Normal variable: writing the value 42 directly on the whiteboard. When you erase the whiteboard, it is gone.

Pointer: writing an address on the whiteboard that tells you which warehouse shelf to visit. Erasing the whiteboard loses the address, but the warehouse item stays put.

Reference: drawing an arrow from a nickname on the whiteboard back to an existing entry on the same whiteboard (or an existing warehouse address). No new content — just a shortcut label.

In one sentence: stack frames hold local names and small values, the heap holds anything dynamic or persistent, and pointers/references are the mechanisms that let stack-frame code reach into the heap or share slots without copying data.

— *End of Notes* —